

Tuyển tập Bài tập: Cây Chỉ số Nhị phân (Fenwick Tree / Binary Indexed Tree - BIT)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Danh mục Bài tập Tuyển chọn

STT	Tên bài toán	Nguồn	Cấp độ	Kỹ thuật trọng tâm	Độ phức tạp
1	Dynamic Range Sum Queries	CSES 1648	● Cơ bản	Fenwick Tree PURQ chuẩn mực	$O((N + Q) \log N)$
2	Range Update Queries	CSES 1651	● Cơ bản	Fenwick Tree RUPQ với mảng hiệu	$O((N + Q) \log N)$
3	Inversion Count	SPOJ INVCNT	● Cơ bản	Đếm số nghịch thế bằng Fenwick Tree	$O(N \log N)$
4	Range Xor Query	AtCoder ABC 185F	● Cơ bản	Fenwick Tree trên nhóm Abel phép XOR	$O((N + Q) \log N)$
5	Greg and Array	Codeforces 296C	● Cơ bản	Mảng hiệu 2 tầng kết hợp Fenwick	$O((N + M + K) \log N)$
6	Rooks Defenders	Codeforces 1679C	● Trung cấp	Hai cây Fenwick hàng & cột kiểm tra chiều	$O((N + Q) \log N)$
7	Distinct Values Queries	CSES 1734	● Trung cấp	Offline Queries + Fenwick cập nhật vị trí cũ	$O((N + Q) \log N)$
8	Distinct Characters Queries	Codeforces 1234D	● Trung cấp	26 cây Fenwick độc lập cho từng ký tự	$O(Q \cdot 26 \log N)$
9	Nested Segments	Codeforces 652D	● Trung cấp	Nén tọa độ + Fenwick đếm đoạn bao nhau	$O(N \log N)$
10	Matrix Summation	SPOJ MATSUM	● Trung cấp	2D Fenwick Tree cập nhật điểm & truy vấn HCN	$O(Q \log N \log M)$

STT	Tên bài toán	Nguồn	Cấp độ	Kỹ thuật trọng tâm	Độ phức tạp
11	Horrible Queries	SPOJ HORRIBLE	● Trung cấp	Fenwick RURQ với 2 cây B_1 và B_2 song song	$O((N + Q) \log N)$
12	Petya and Array	Codeforces 1042D	● Nâng cao	Nén tọa độ tiền tố + Fenwick đếm đoạn con $< T$	$O(N \log N)$
13	Multiset	Codeforces 1354D	● Nâng cao	Binary Lifting on BIT (Order Statistic) bộ nhớ 28MB	$O((N + Q) \log N)$
14	Fixed Point Removal	Codeforces 1405E	● Nâng cao	Nhảy nhị phân trên Fenwick Tree + Offline	$O((N + Q) \log N)$
15	Nested Ranges Count	CSES 2169	● Nâng cao	Đếm số khoảng bao và bị bao hai chiều	$O(N \log N)$
16	Ball	Codeforces 12D	● Nâng cao	Nén tọa độ 3 chiều (CDQ / Fenwick Max Query)	$O(N \log N)$

● Phần 1: Cấp độ Cơ bản & Khởi động (Warm-up)

Bài 1: Dynamic Range Sum Queries (CSES 1648)

- **Đề bài:** Cho mảng N phần tử. Có Q truy vấn thuộc 2 loại: (1) Cập nhật giá trị tại vị trí k thành u ; (2) Tính tổng các phần tử trong đoạn $[a, b]$.
- **Ràng buộc:** $1 \leq N, Q \leq 2 \cdot 10^5$, giá trị phần tử $\leq 10^9$.
- **Gợi ý:**
- *Hint 1:* Cài đặt Fenwick Tree cơ bản (PURQ) với mảng kiểu `long long`.
- *Hint 2:* Khi gán $A[k] = u$, lượng biến thiên là $\Delta = u - A[k]$. Gọi `update(k, \Delta)` và gán $A[k] = u$.
- *Hint 3:* Truy vấn tổng đoạn $[a, b]$ bằng `query(b) - query(a - 1)`.

Bài 2: Range Update Queries (CSES 1651)

- **Đề bài:** Cho mảng N phần tử. Có Q truy vấn: (1) Cộng thêm u vào tất cả phần tử trong đoạn $[a, b]$; (2) In ra giá trị của phần tử tại vị trí k .
- **Ràng buộc:** $1 \leq N, Q \leq 2 \cdot 10^5$.
- **Gợi ý:**
- *Hint 1:* Áp dụng kỹ thuật Mảng hiệu: $D[i] = A[i] - A[i - 1]$. Khi đó giá trị $A[k] = \sum_{i=1}^k D[i]$.
- *Hint 2:* Cộng u vào $[a, b]$: `update(a, u)` và `update(b + 1, -u)` trên Fenwick Tree quản lý mảng hiệu D .
- *Hint 3:* Truy vấn giá trị tại k chỉ là một thao tác `query(k)` trong $O(\log N)$.

Bài 3: Inversion Count (SPOJ INVCNT)

- **Đề bài:** Cho một dãy số A gồm N số nguyên dương phân biệt. Đếm số cặp nghịch thế, tức là số cặp chỉ số (i, j) sao cho $i < j$ và $A[i] > A[j]$.
 - **Ràng buộc:** $1 \leq N \leq 2 \cdot 10^5$, $A[i] \leq 10^7$.
 - **Gợi ý:**
 - *Hint 1:* Duyệt mảng từ phải sang trái (từ N về 1). Khi đứng tại $A[i]$, số phần tử đứng sau $A[i]$ và nhỏ hơn $A[i]$ chính là tổng tiền tố tần số các số trong đoạn $[1, A[i] - 1]$.
 - *Hint 2:* Sử dụng Fenwick Tree làm mảng đếm tần số: Gọi `query(A[i] - 1)` cộng vào đáp án tổng số nghịch thế.
 - *Hint 3:* Sau đó đánh dấu đã gặp $A[i]$ bằng cách `update(A[i], 1)`. Tổng số nghịch thế có thể lên tới $\binom{N}{2} \approx 2 \cdot 10^{10}$, dùng kiểu `long long`.
-

Bài 4: Range Xor Query (AtCoder ABC 185F)

- **Đề bài:** Cho mảng N phần tử. Có 2 loại truy vấn: (1) $A[X] \leftarrow A[X] \oplus Y$; (2) Tính tổng XOR của các phần tử từ X đến Y : $A[X] \oplus A[X + 1] \oplus \dots \oplus A[Y]$.
 - **Ràng buộc:** $1 \leq N, Q \leq 3 \cdot 10^5$.
 - **Gợi ý:**
 - *Hint 1:* Phép toán XOR ($\oplus$) có tính chất kết hợp, giao hoán và tự triệt tiêu ($X \oplus X = 0$). Phép toán ngược của XOR chính là phép XOR!
 - *Hint 2:* Trong Fenwick Tree, thay toàn bộ phép cộng `+=` bằng phép XOR `^=`.
 - *Hint 3:* Truy vấn XOR đoạn $[L, R]$ tính bằng `query(R) ^ query(L - 1)`.
-

Bài 5: Greg and Array (Codeforces 296C)

- **Đề bài:** Cho mảng N phần tử. Có M thao tác: thao tác i cộng d_i vào đoạn $[l_i, r_i]$. Cho K truy vấn: mỗi truy vấn yêu cầu thực hiện lại các thao tác từ thứ tự x đến y . In ra mảng cuối cùng.
 - **Ràng buộc:** $1 \leq N, M, K \leq 10^5$.
 - **Gợi ý:**
 - *Hint 1:* Dùng mảng hiệu tăng 1 để đếm xem thao tác thứ i ($1 \leq i \leq M$) được thực hiện tổng cộng bao nhiêu lần sau K truy vấn.
 - *Hint 2:* Sau khi tính được số lần thực hiện $cnt[i]$ của mỗi thao tác, thao tác i tương đương với việc cộng $cnt[i] \times d_i$ vào đoạn $[l_i, r_i]$.
 - *Hint 3:* Dùng mảng hiệu tăng 2 (hoặc Fenwick RUPQ) để áp dụng các thay đổi lên mảng ban đầu trong $O(N + M + K)$.
-

● Phần 2: Cấp độ Trung cấp — Kỹ thuật Two-star Cốt lõi

Bài 6: Rooks Defenders (Codeforces 1679C)

- **Đề bài:** Trên bàn cờ vua $N \times N$, có 3 loại truy vấn: (1) Đặt một quân xe vào ô (r, c) ; (2) Nhắc quân xe khỏi ô (r, c) ; (3) Kiểm tra xem tất cả các ô trong hình chữ nhật con $[x_1, y_1] \rightarrow [x_2, y_2]$ có bị ít nhất một quân xe chiếu hay không?
- **Ràng buộc:** $1 \leq N \leq 10^5, 1 \leq Q \leq 2 \cdot 10^5$.
- **Gợi ý:**
- *Hint 1:* Một hình chữ nhật con bị chiếu hoàn toàn khi và chỉ khi: Tất cả các hàng từ x_1 đến x_2 đều có ít nhất 1 quân xe, HOẶC tất cả các cột từ y_1 đến y_2 đều có ít nhất 1 quân xe.
- *Hint 2:* Duy trì mảng đếm số xe trên từng hàng `row_cnt[r]` và từng cột `col_cnt[c]`.
- *Hint 3:* Dùng 2 cây Fenwick: `BIT_row[r] = 1` nếu hàng r có ≥ 1 xe, ngược lại 0. Tương tự cho cột. Điều kiện hàng thỏa mãn khi `BIT_row.query_range(x1, x2) == x2 - x1 + 1`.

Bài 7: Distinct Values Queries (CSES 1734)

- **Đề bài:** Cho mảng N phần tử. Có Q truy vấn: Đếm số lượng phần tử phân biệt trong đoạn $[a, b]$.
- **Ràng buộc:** $1 \leq N, Q \leq 2 \cdot 10^5$.
- **Gợi ý:**
- *Hint 1:* Xử lý Offline: Sắp xếp các truy vấn tăng dần theo đầu mút phải b .
- *Hint 2:* Duyệt i từ 1 đến N . Dùng `std::map` lưu vị trí xuất hiện gần nhất của giá trị $A[i]$ là `last_pos`.
- *Hint 3:* Nếu $A[i]$ đã từng xuất hiện trước đó tại `last_pos`, ta xóa đóng góp cũ: `update(last_pos, -1)`. Sau đó thêm đóng góp mới tại vị trí hiện tại: `update(i, 1)`. Đáp án cho truy vấn $[a, b]$ chính là `query_range(a, b)`.

Bài 8: Distinct Characters Queries (Codeforces 1234D)

- **Đề bài:** Cho một xâu ký tự S độ dài N . Có Q truy vấn: (1) Thay đổi ký tự tại vị trí pos thành c ; (2) Đếm số ký tự phân biệt trong đoạn con $S[l \dots r]$.
- **Ràng buộc:** $1 \leq N, Q \leq 10^5$, các ký tự là chữ cái thường tiếng Anh ('a' đến 'z').
- **Gợi ý:**
- *Hint 1:* Bảng chữ cái chỉ có đúng 26 ký tự. Ta có thể dựng **26 cây Fenwick độc lập**: `BIT[c]` lưu vị trí các ký tự c .
- *Hint 2:* Thao tác đổi ký tự: Trừ 1 tại ký tự cũ và cộng 1 tại ký tự mới trên cây tương ứng.
- *Hint 3:* Truy vấn: Duyệt c từ 'a' đến 'z'. Ký tự c xuất hiện trong đoạn $[l, r]$ khi và chỉ khi `BIT[c].query_range(1, r) > 0`. Đếm số lượng ký tự thỏa mãn trong $O(26 \log N)$.

Bài 9: Nested Segments (Codeforces 652D)

- **Đề bài:** Cho N đoạn thẳng $[l_i, r_i]$ trên trục số. Với mỗi đoạn thẳng, đếm xem nó chứa hoàn toàn bao nhiêu đoạn thẳng khác trong tập hợp đã cho.

- **Ràng buộc:** $1 \leq N \leq 2 \cdot 10^5$, tọa độ $|l_i|, |r_i| \leq 10^9$.
- **Gợi ý:**
- *Hint 1:* Đoạn j nằm trong đoạn i khi và chỉ khi $l_i \leq l_j$ và $r_j \leq r_i$.
- *Hint 2:* Nén tọa độ toàn bộ các giá trị l và r về khoảng $[1, 2N]$. Sắp xếp các đoạn thẳng giảm dần theo l_i .
- *Hint 3:* Khi duyệt theo l giảm dần, mọi đoạn đã xét trước đó đều có $l_j \geq l_i$. Ta chỉ cần đếm xem trong các đoạn đã xét có bao nhiêu đoạn có $r_j \leq r_i$, chính là thao tác `query(r_i)` trên Fenwick Tree! Sau đó gọi `update(r_i, 1)`.

Bài 10: Matrix Summation (SPOJ MATSUM)

- **Đề bài:** Cho ma trận $N \times N$ ban đầu toàn 0. Có các thao tác: `SET x y val` (gán ô $(x, y) = val$) và `SUM x1 y1 x2 y2` (tính tổng hình chữ nhật con).
- **Ràng buộc:** $1 \leq N \leq 1024$, số thao tác $\leq 2 \cdot 10^5$.
- **Gợi ý:**
- *Hint 1:* Cài đặt 2D Fenwick Tree.
- *Hint 2:* Với `SET x y val`, tính độ chênh lệch $\Delta = val - get(x, y)$ và gọi `update2D(x + 1, y + 1, \Delta)` (chú ý đổi về 1-indexed).
- *Hint 3:* Với `SUM x1 y1 x2 y2`, áp dụng công thức bù trừ 2D với 4 lần gọi `query_prefix` trong $O(\log N \log N)$.

Bài 11: Horrible Queries (SPOJ HORRIBLE)

- **Đề bài:** Cho mảng N phần tử ban đầu toàn 0. Thực hiện 2 loại truy vấn: (1) Cộng v vào tất cả phần tử trong đoạn $[p, q]$; (2) Tính tổng các phần tử trong đoạn $[p, q]$.
- **Ràng buộc:** $1 \leq N, Q \leq 10^5, v \leq 10^7$.
- **Gợi ý:**
- *Hint 1:* Áp dụng kỹ thuật Fenwick Cấp độ 3 (Range Update, Range Query - RURQ).
- *Hint 2:* Duy trì 2 cây Fenwick B_1 và B_2 . Tổng tiền tố là $(p + 1) \cdot B_1.query(p) - B_2.query(p)$.
- *Hint 3:* Tất cả các mảng đều dùng kiểu `long long` để tránh tràn số.

● Phần 3: Cấp độ Nâng cao & Olympic (Advanced ICPC/OLP)

Bài 12: Petya and Array (Codeforces 1042D)

- **Đề bài:** Cho mảng N phần tử và số T . Đếm số cặp chỉ số (l, r) với $l \leq r$ sao cho tổng đoạn con $\sum_{i=l}^r a_i < T$.
- **Ràng buộc:** $1 \leq N \leq 2 \cdot 10^5, |a_i| \leq 10^9, |T| \leq 2 \cdot 10^{14}$.
- **Gợi ý:**
- *Hint 1:* Gọi $pref[i]$ là tổng tiền tố của i phần tử đầu tiên ($pref[0] = 0$). Điều kiện trở thành: $pref[r] - pref[l - 1] < T \iff pref[l - 1] > pref[r] - T$.
- *Hint 2:* Khi cố định r , ta cần đếm số vị trí $l - 1 < r$ thỏa mãn $pref[l - 1] > pref[r] - T$.

- *Hint 3*: Nén tọa độ toàn bộ các giá trị $pref[i]$ và $pref[i] - T$. Duyệt r từ 0 đến N , dùng Fenwick Tree để đếm số phần tử lớn hơn ngưỡng trong $O(\log N)$.

Bài 13: Multiset (Codeforces 1354D)

- **Đề bài**: Duy trì một đa tập hợp số nguyên. Cho N phần tử ban đầu và Q truy vấn: Thêm số k vào tập, hoặc xóa phần tử nhỏ thứ k khỏi tập. Giới hạn bộ nhớ cực kỳ khắt khe: **chỉ có 28 MB**!
- **Ràng buộc**: $1 \leq N, Q \leq 10^6$. Giá trị các số $\leq 10^6$.
- **Gợi ý**:
- *Hint 1*: Segment Tree cần $4 \times 10^6 \times 4 \text{ bytes} \approx 16 \text{ MB}$, dễ bị MLE nếu không cẩn thận.
`std::multiset` hay Treap sẽ quá bộ nhớ ngay lập tức.
- *Hint 2*: Fenwick Tree chỉ tốn đúng $10^6 \times 4 \text{ bytes} \approx 4 \text{ MB}$, hoàn toàn nằm gọn trong 28 MB!
- *Hint 3*: Thao tác tìm và xóa phần tử thứ k : Dùng **Binary Lifting on Fenwick Tree** trong $O(\log N)$ để tìm trực tiếp giá trị cần xóa mà không dùng chèn nhị phân hai lần ($O(\log^2 N)$).

Bài 14: Fixed Point Removal (Codeforces 1405E)

- **Đề bài**: Cho mảng N phần tử. Một phần tử a_i được gọi là "điểm cố định" nếu $a_i = i$. Khi có điểm cố định, ta có thể xóa nó và các phần tử bên phải dồn sang trái. Với mỗi truy vấn (x, y) , nếu ta không được xóa x phần tử đầu và y phần tử cuối, hỏi số lượng phần tử tối đa có thể xóa được là bao nhiêu?
- **Ràng buộc**: $1 \leq N, Q \leq 3 \cdot 10^5$.
- **Gợi ý**:
- *Hint 1*: Đặt $d_i = i - a_i$. Nếu $d_i < 0$, phần tử này không bao giờ xóa được. Nếu $d_i \geq 0$, để xóa được nó thì trước nó cần bị xóa ít nhất d_i phần tử.
- *Hint 2*: Với mỗi phần tử i , ta tìm vị trí pos xa nhất về bên trái sao cho nếu bắt đầu từ pos , phần tử i vẫn có thể bị xóa.
- *Hint 3*: Tìm pos bằng Binary Lifting trên Fenwick Tree. Sau đó xử lý Offline các truy vấn theo đầu mút phải $N - y$ trong $O((N + Q) \log N)$.

Bài 15: Nested Ranges Count (CSES 2169)

- **Đề bài**: Cho N đoạn thẳng $[x_i, y_i]$. Với mỗi đoạn thẳng, tính: (1) Số lượng đoạn thẳng khác chứa nó; (2) Số lượng đoạn thẳng khác bị nó chứa.
- **Ràng buộc**: $1 \leq N \leq 2 \cdot 10^5$, tọa độ $\leq 10^9$.
- **Gợi ý**:
- *Hint 1*: Nén tọa độ các đầu mút y_i . Sắp xếp các đoạn tăng dần theo x ; nếu cùng x thì giảm dần theo y .
- *Hint 2*: Để đếm số đoạn chứa đoạn i : Duyệt xuôi từ 1 đến N , mọi đoạn trước đó đều có $x_{prev} \leq x_i$. Điều kiện chứa là $y_{prev} \geq y_i$, truy vấn số lượng đoạn có $y \geq y_i$ bằng Fenwick Tree.
- *Hint 3*: Để đếm số đoạn bị đoạn i chứa: Duyệt ngược từ N về 1 và truy vấn tương tự với $y \leq y_i$.

Bài 16: Ball (Codeforces 12D)

- **Đề bài:** Có N quý cô, mỗi quý cô có 3 chỉ số sắc đẹp, tài năng, giàu có (B_i, I_i, R_i) . Một quý cô sẽ tự sát nếu tồn tại một quý cô khác vượt trội hơn cô ấy ở **cả 3 chỉ số**. Đếm số quý cô tự sát.
- **Ràng buộc:** $1 \leq N \leq 5 \cdot 10^5$.
- **Gợi ý:**
 - *Hint 1:* Bài toán tìm kiếm điểm bị trội 3 chiều (3D Dominance).
 - *Hint 2:* Khử chiều thứ nhất: Sắp xếp các quý cô giảm dần theo B_i .
 - *Hint 3:* Khử chiều thứ hai & ba: Nén tọa độ I_i . Dùng Fenwick Tree quản lý giá trị **lớn nhất** của R tại các vị trí I : `bit[I]` lưu $\max(R)$. Khi xét một quý cô, truy vấn giá trị R lớn nhất trong các quý cô có chỉ số $I > I_i$. Nếu giá trị này $> R_i \implies$ cô ấy tự sát!



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.